

Control Flow — if, elif, else

Python Beginner's Guide

Learning Objectives

After completing this chapter, you will be able to:

- Explain what control flow means.
- Use if statements to make decisions.
- Use else for an alternative action.
- Use elif to test multiple conditions.
- Use comparison and logical operators in conditions.
- Create nested if statements.

1. What Is Control Flow?

Control flow describes the order in which instructions in a program are executed. Normally, Python runs statements from top to bottom. Control-flow statements allow a program to make decisions and choose which instructions should run.

For example, a program can check whether a person's age is 18 or more and display a different message depending on the result.

2. The if Statement

An if statement runs a block of code only when its condition is True. Python uses indentation to show which statements belong to the if block.

Basic syntax:

```
if condition:
    # code to run
```

Example:

```
age = 20

if age >= 18:
    print("You are an adult.")
```

3. Comparison Operators

Comparison operators are commonly used to create conditions.

- == Equal to
- != Not equal to
- > Greater than

- < Less than
- >= Greater than or equal to
- <= Less than or equal to

Examples:

```
x = 10
print(x == 10)
print(x > 5)
print(x < 5)
```

4. The if-else Statement

Use else when you want one block to run if the if condition is False.

Example:

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

5. The elif Statement

elif means 'else if'. It lets you check another condition when the previous condition was False. You can use multiple elif blocks.

Example:

```
marks = 75

if marks >= 80:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Needs improvement")
```

6. Logical Operators

Logical operators combine or modify conditions.

- **and** — True only when both conditions are True.
- **or** — True when at least one condition is True.
- **not** — Reverses a Boolean condition.

Examples:

```
age = 25
has_id = True
```

```
if age >= 18 and has_id:  
    print("Access allowed")
```

7. Using and

The and operator requires every condition connected by it to be True.

```
temperature = 25  
  
if temperature >= 20 and temperature <= 30:  
    print("Comfortable temperature")
```

8. Using or

The or operator runs a condition when at least one of its conditions is True.

```
day = "Saturday"  
  
if day == "Saturday" or day == "Sunday":  
    print("Weekend")
```

9. Using not

The not operator reverses a Boolean value or condition.

```
is_raining = False

if not is_raining:
    print("You can go outside.")
```

10. Nested if Statements

A nested if statement is an if statement inside another if statement. Nested conditions are useful when one decision depends on another.

Example:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
    else:
        print("Entry not allowed")
```

11. Indentation in Python

Indentation is very important in Python. It tells Python which statements belong to a block. The commonly used style is four spaces for each level of indentation.

Correct:

```
if age >= 18:
    print("Adult")
```

Incorrect indentation can cause an IndentationError or change how the program works.

12. Conditions with Strings

Conditions can also compare strings.

```
password = "python123"

if password == "python123":
    print("Correct password")
else:
    print("Incorrect password")
```

13. Checking Multiple Conditions

You can combine comparison and logical operators to create more useful decisions.

```
username = "admin"
password = "1234"
```

```
if username == "admin" and password == "1234":  
    print("Login successful")  
else:  
    print("Login failed")
```

14. User Input with Conditions

The `input()` function can be combined with `if` statements. Remember that `input()` returns a string, so numeric input usually needs conversion.

```
age = int(input("Enter your age: "))  
  
if age >= 18:  
    print("You can apply.")  
else:  
    print("You are too young to apply.")
```

15. Mini Project — Simple Grade Checker

Create a program that asks for marks and displays a grade based on the marks.

Example solution:

```
marks = float(input("Enter your marks: "))  
  
if marks >= 80:  
    print("Grade A")  
elif marks >= 70:  
    print("Grade B")  
elif marks >= 60:  
    print("Grade C")  
elif marks >= 50:  
    print("Grade D")  
else:  
    print("Fail")
```

16. Practice Exercises

Try these exercises on your own:

- 1 Write a program that checks whether a number is positive or negative.
- 2 Check whether a number is even or odd.
- 3 Ask for a user's age and display an appropriate message.
- 4 Create a program that compares two numbers and prints the larger one.
- 5 Create a simple login checker using a username and password.
- 6 Create a temperature checker using `if`, `elif`, and `else`.

17. Common Mistakes

- Using = instead of == when comparing values.
- Forgetting the colon (:) after if, elif, or else.
- Incorrect indentation.
- Forgetting to convert numeric input with int() or float().
- Writing conditions in an order that prevents a later condition from being reached.

18. Chapter Summary

Control flow lets Python programs make decisions. The if statement runs code when a condition is True. else provides an alternative block, while elif allows multiple conditions to be tested. Comparison operators and logical operators make conditions more powerful. Correct indentation is essential because Python uses indentation to define code blocks.

Quick Review

- if checks a condition.
- elif checks another condition when previous conditions are False.
- else runs when all preceding conditions are False.
- == compares two values for equality.
- and, or, and not are logical operators.
- Python uses indentation to define blocks.

End of Chapter